

AlgebraicDynamics discrete SIR interventions

Difference equations, FunctionMap, lockdown, and vaccination

This vignette represents the same SIR intervention story with discrete-time `AlgebraicDynamics.jl` resource sharers. It follows the [epirecipes/sir-julia](#) `FunctionMap` example: a rate is converted to a per-step transition probability by $1 - \exp(-r * t)$, with $t = 0.1$.

The model is discrete in time and continuous in state. It is not expected to match the continuous-time ODE solution exactly; instead, it is checked against a direct `DiscreteProblem(..., FunctionMap())` implementation of the same difference equations.

This vignette has its own local `Project.toml` because `AlgebraicDynamics.jl` currently resolves against older `Catlab.jl` and `DiffEqCallbacks.jl` versions than the main vignette environment.

```
using AlgebraicDynamics
using Catlab.Programs
using Catlab.WiringDiagrams: oapply
using CategoricalInterventions
using ComponentArrays
using DiffEqCallbacks
using OrdinaryDiffEq
using Plots

default(; linewidth=2, grid=false)

rate_to_proportion(r, t) = 1 - exp(-r * t)
```

`rate_to_proportion` (generic function with 1 method)

A composed discrete SIR model

`DiscreteResourceSharer` expects a one-step update rule $u[n+1] = f(u[n], p, t)$. The infection primitive updates the local S , I resources, while the recovery primitive updates the local I ,

R resources. AlgebraicDynamics composes these primitives by summing their state changes along the shared I resource.

```
function infection_update(u, p, t)
    S, I = u
    infection = rate_to_proportion(p.inf * I / p.N, p.t) * S
    return [S - infection, I + infection]
end

function recovery_update(u, p, t)
    I, R = u
    recovery = rate_to_proportion(p.rec, p.t) * I
    return [I - recovery, R + recovery]
end

infection = DiscreteResourceSharer{Float64}(2, infection_update)
recovery = DiscreteResourceSharer{Float64}(2, recovery_update)

sir_pattern = @relation (S, I, R) begin
    infection(S, I)
    recovery(I, R)
end

sir_system = oapply(sir_pattern, [infection, recovery])
```

The state vector produced by this composition is positional, with order S, I, R. Parameters are labelled with ComponentArrays.jl. The fixed population N is valid here because the model flows and vaccination pulse preserve total population.

```
u0 = [990.0, 10.0, 0.0]
p0 = ComponentArray{inf=0.05 * 10.0, rec=0.25, N=sum(u0), t=0.1}
tspan = (0.0, 40.0)
save_grid = 0.0:p0.t:40.0

prob = DiscreteProblem(sir_system, u0, tspan, p0)
baseline = solve(prob, FunctionMap(); dt=p0.t, saveat=save_grid)

baseline.retcode, length(baseline.t), baseline[3, end], sum(baseline.u[end])
```

```
(SciMLBase.ReturnCode.Success, 401, 782.3831245026182, 999.9999999999995)
```

Interventions

The parameter intervention scales `:inf`, which represents $\beta \cdot c$. The state intervention is a discrete-time vaccination pulse at $t = 15$: remove 50 from S and add 50 to R. The pulse interval has one discrete step of width Δt .

```
indexing = PetriIndexing(
    Dict(:inf => :inf, :rec => :rec),
    Dict(:S => 1, :I => 2, :R => 3),
)

inf_rate = Target(
    :inf;
    kind=Parameter,
    value_type=Float64,
    algebra=MultiplicativeAlgebra(),
)

lockdown = InterventionProgram(
    InterventionAtom(:lockdown, inf_rate, Interval(5.0, 25.0), Scale(0.5)),
)

susceptible = Target(:S; kind=State, value_type=Float64, algebra=AdditiveAlgebra())
recovered = Target(:R; kind=State, value_type=Float64, algebra=AdditiveAlgebra())

vaccination = InterventionProgram(
    InterventionAtom(:vaccinate_from_s, susceptible, Interval(15.0, 15.0 + p0.t), Add(-50.0)),
    InterventionAtom(:vaccinate_to_r, recovered, Interval(15.0, 15.0 + p0.t), Add(50.0)),
)

combined = compose_interventions(lockdown, vaccination)
```

`InterventionProgram(3 atoms)`

`FunctionMap` accepts callbacks, so the same lowering used in the continuous vignettes can mutate `integrator.p` for parameter intervals and `integrator.u` for state pulses.

```
lockdown_sol = solve(
    prob,
    FunctionMap();
    dt=p0.t,
```

```

        callback=to_callback(lockdown, indexing; baseline_p=p0),
        saveat=save_grid,
    )

vaccination_sol = solve(
    prob,
    FunctionMap();
    dt=p0.t,
    callback=to_callback(vaccination, indexing),
    saveat=save_grid,
)

combined_sol = solve(
    prob,
    FunctionMap();
    dt=p0.t,
    callback=to_callback(combined, indexing; baseline_p=p0),
    saveat=save_grid,
)

(
    baseline_R = baseline[3, end],
    lockdown_R = lockdown_sol[3, end],
    vaccination_R = vaccination_sol[3, end],
    combined_R = combined_sol[3, end],
    combined_total = sum(combined_sol.u[end]),
)

```

```
(baseline_R = 782.3831245026182, lockdown_R = 362.770280153676, vaccination_R = 780.196102307)
```

Cross-check against a direct FunctionMap

The direct difference equation mirrors the epirecipes FunctionMap example. This checks that the composed AlgebraicDynamics model and the direct map are the same discrete SIR model.

```

function sir_map!(du, u, p, t)
    S, I, R = u
    infection = rate_to_proportion(p.inf * I / p.N, p.t) * S
    recovery = rate_to_proportion(p.rec, p.t) * I
    du[1] = S - infection
    du[2] = I + infection - recovery
end

```

```

    du[3] = R + recovery
    return nothing
end

reference_prob = DiscreteProblem(sir_map!, copy(u0), tspan, p0)

reference_baseline = solve(reference_prob, FunctionMap(); dt=p0.t, saveat=save_grid)
reference_combined = solve(
    reference_prob,
    FunctionMap();
    dt=p0.t,
    callback=to_callback(combined, indexing; baseline_p=p0),
    saveat=save_grid,
)

max_reference_difference = maximum(abs, Array(baseline) .- Array(reference_baseline))
max_combined_difference = maximum(abs, Array(combined_sol) .- Array(reference_combined))

max_reference_difference, max_combined_difference

```

```
(5.684341886080802e-14, 1.4779288903810084e-12)
```

```
max_reference_difference < 1e-9 && max_combined_difference < 1e-9
```

```
true
```

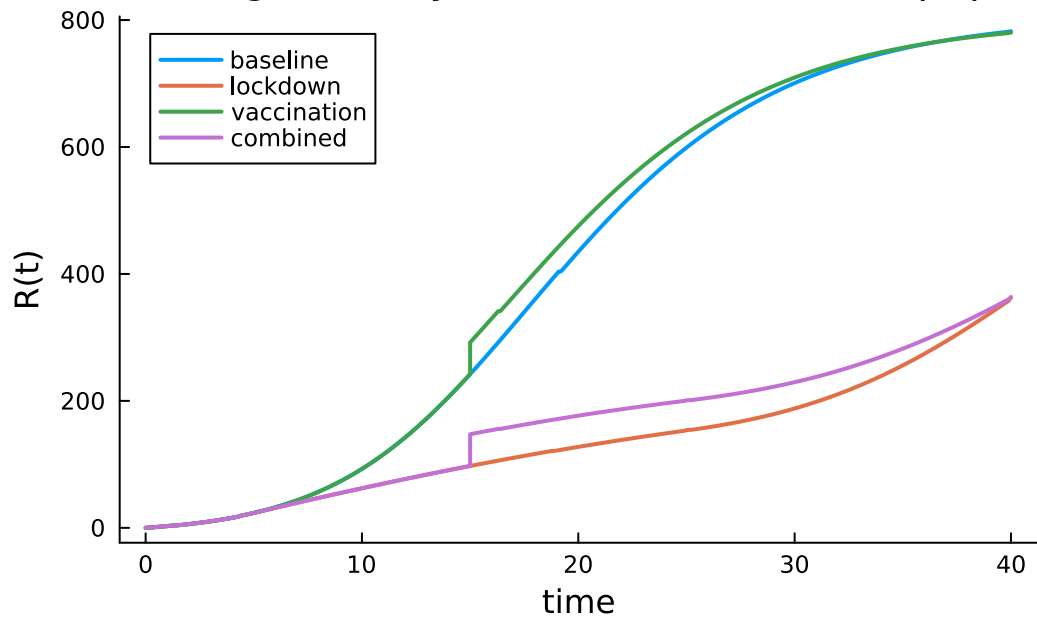
Plotting

```

plot(
    baseline.t,
    baseline[3, :];
    label="baseline",
    xlabel="time",
    ylabel="R(t)",
    title="Discrete AlgebraicDynamics SIR recovered population",
)
plot!(lockdown_sol.t, lockdown_sol[3, :]; label="lockdown")
plot!(vaccination_sol.t, vaccination_sol[3, :]; label="vaccination")
plot!(combined_sol.t, combined_sol[3, :]; label="combined")

```

Discrete AlgebraicDynamics SIR recovered popula



```
plot(  
    combined_sol.t,  
    combined_sol[1, :];  
    label="S",  
    xlabel="time",  
    ylabel="population",  
    title="Discrete combined lockdown and vaccination",  
)  
plot!(combined_sol.t, combined_sol[2, :]; label="I")  
plot!(combined_sol.t, combined_sol[3, :]; label="R")
```

